

AI Usage - DISCLOSURE FORM

1. Team Details

Team Name: TIET_Agentic_Alchemists

Project / Product Name: FixPilot based on Theme 2 — Smart Guided Troubleshooting Engine

Institution: Thapar Institute of Engineering & Technology

Submission Date: 29th September, 2026

2. AI Usage Declaration

Did your team use any Artificial Intelligence (AI) in developing this project?

Yes — details are given below.

3. Purpose of AI Usage (Brief Details)

Idea generation / brainstorming: Human-led. The human (Armaan) defined the product direction, the deterministic-first architecture, and the provider choice (Google Gemini, for zero cost and the FAQ bonus points). The AI assistant contributed implementation options and trade-off analysis on request.

Code generation or assistance: Yes — primary use. The AI coding assistant generated the application code, tests, and evaluation scripts under the human's direction; the human reviewed, tested, and requested corrections.

UI / UX design: Not applicable — the deliverable is a headless REST API (no user interface).

Content creation: Yes — README, submission notes, evaluation reports, and this disclosure draft were drafted by the AI assistant and reviewed by the human.

Data analysis: Yes — the AI assistant wrote and ran the evaluation harness (metrics, latency percentiles, ablation comparisons); the human defined the gates and interpreted the results.

Testing / debugging: Yes — the AI assistant wrote the pytest suite (91 tests), reproduced failures, and fixed bugs found during development and live integration testing.

Other: —

4. Feature Origin Classification

Classification key: Self-Generated = written by the human alone; AI-Generated = written by AI alone; Both = human-directed, AI-implemented, human-reviewed. Nearly every feature below is "Both": the human set requirements, architecture, and provider choices, and the AI coding

assistant (Muse, built by Meta) produced the implementation, which the human reviewed and corrected.

1. Pipeline architecture & deterministic-first design

Classification: Both

AI tools / platform used: Muse (Meta) AI coding assistant for implementation; Google Gemini API (free tier) for LLM enrichment features.

Prompt summary: Human: "build a troubleshooting engine that works with zero LLM cost — rules first, LLM only for language enrichment, graceful fallback if the LLM is unavailable."

Output summary: Staged pipeline (enrich → extract → retrieve → validate → cache) with PIPELINE_MODE auto/rules/llm; deterministic defaults satisfy every hard gate at \$0/query.

Human modifications / review: Human chose deterministic-first over a pure-LLM design; reviewed stage boundaries and the fallback/degraded-mode behaviour.

2. LLM integration layer (Gemini OpenAI-compatible client)

Classification: Both

AI tools / platform used: Muse (Meta) AI coding assistant for implementation; Google Gemini API (free tier) for LLM enrichment features.

Prompt summary: Human: "use Gemini API key instead of Muse; no Anthropic keys — rewire everything; keep structured output with a JSON fallback and log tokens/cost."

Output summary: app/llm.py: LLMClient over Gemini's OpenAI-compatible endpoint (chat.completions.parse with Pydantic models → JSON-mode fallback + local validation), retries, per-call token/cost logging, Gemini pricing table.

Human modifications / review: Human directed two provider switches (Anthropic → Meta Muse Spark → Google Gemini, for cost and FAQ bonus points); default model updated gemini-2.5-flash → gemini-3.8-flash after Google retired 2.5 for new keys; pricing corrected to \$0.75/\$3.75 per 1M.

3. Rule-based query enrichment & paraphrase generation

Classification: Both

AI tools / platform used: Muse (Meta) AI coding assistant for implementation; Google Gemini API (free tier) for LLM enrichment features.

Prompt summary: Human: "paraphrases must be deterministic and diverse — formal, casual, keyword-only, frustrated, typos — 8–10 per query for the cache and for results.jsonl scoring."

Output summary: rules_enrich(): slot extraction + deterministic paraphrase templates (seeded RNG) producing 8–10 unique variations per query; verified across all 20 queries.

Human modifications / review: Human required the 8–10 count (official scoring: 5 pts) and added results.jsonl query_variations wiring plus a regression test.

4. Step extraction with source-span grounding

Classification: Both

AI tools / platform used: Muse (Meta) AI coding assistant for implementation; Google Gemini API (free tier) for LLM enrichment features.

Prompt summary: Human: "steps must be grounded in the supplied SITS text — no hallucinated steps."

Output summary: Extractor that drafts steps then aligns each step to source spans in the SITS payload; ungrounded steps are dropped or flagged.

Human modifications / review: Human tightened grounding rules during review; added regression tests for span alignment.

5. Hybrid deeplink retrieval (BM25 + dense + cross-encoder)

Classification: Both

AI tools / platform used: Muse (Meta) AI coding assistant for implementation; Google Gemini API (free tier) for LLM enrichment features.

Prompt summary: Human: "deeplinks must be copied verbatim from the 578-entry catalog — never invent one, never leak a URL."

Output summary: Three-stage retrieval (BM25 → dense embeddings → cross-encoder rerank) selecting catalog deeplinks; URL scrubbing on all outputs.

Human modifications / review: Human set the verbatim/no-URL-leak constraints; reviewed ranking behaviour on edge queries.

6. Validators (schema, URL scrubbing, catalog compliance)

Classification: Both

AI tools / platform used: Muse (Meta) AI coding assistant for implementation; Google Gemini API (free tier) for LLM enrichment features.

Prompt summary: Human: "make violations impossible to ship — schema, catalog, and URL-leak checks as hard gates."

Output summary: compliance.violations() + validators: ContextDeeplinkResponse schema check, URL scrub, catalog-membership check; eval exits non-zero on any violation.

Human modifications / review: Human defined the gate list from the official spec (schema, no URL leaks, catalog compliance, latency).

7. Multi-vector paraphrase cache (FAISS + SQLite)

Classification: Both

AI tools / platform used: Muse (Meta) AI coding assistant for implementation; Google Gemini API (free tier) for LLM enrichment features.

Prompt summary: Human: "repeat queries and paraphrases must hit cache with P95 $\leq$ 300 ms."

Output summary: SQLite plan store + FAISS vector index over query + paraphrase embeddings; observed cache P95 $\sim$ 194 ms, paraphrase hit rate 97.5%.

Human modifications / review: Human set the 300 ms budget from the official spec; reviewed hit-rate methodology (exact / paraphrase / same-article).

8. REST API (FastAPI)

Classification: Both

AI tools / platform used: Muse (Meta) AI coding assistant for implementation; Google Gemini API (free tier) for LLM enrichment features.

Prompt summary: Human: "expose GET /health and POST /v1/troubleshoot, unauthenticated, matching the official schema."

Output summary: FastAPI app with the two required endpoints, request/response validation against ContextDeeplinkResponse, Dockerfile for reproducible deployment.

Human modifications / review: Human verified endpoint paths and the no-auth requirement against the Theme 2 spec.

9. Evaluation harness & ablations

Classification: Both

AI tools / platform used: Muse (Meta) AI coding assistant for implementation; Google Gemini API (free tier) for LLM enrichment features.

Prompt summary: Human: "prove the gates: schema, latency, deeplink accuracy, cache hits — and ablate rules vs hybrid vs full-LLM."

Output summary: eval/run_eval.py (compliance + latency gates, metrics.json/md), eval/ablations.py (pipeline ablations), eval/paraphrase_gen.py; 91-test pytest suite.

Human modifications / review: Human defined the hard gates from the official materials; requested the live-Gemini ablation run and the rules-only baseline.

10. Docs / README / submission materials

Classification: Both

AI tools / platform used: Muse (Meta) AI coding assistant for implementation; Google Gemini API (free tier) for LLM enrichment features.

Prompt summary: Human: "make it GitHub-worthy and submission-ready: README, env example, submission notes."

Output summary: README.md, SUBMISSION.md, CLAUDE.md, .env.example, regenerated results.jsonl with query_variations, this disclosure draft.

Human modifications / review: Human reviewed docs for accuracy (e.g. corrected model names/pricing after the 2.5 retirement) and owns final sign-off.

5. Ethical & Compliance Confirmation

AI usage complies with the hackathon guidelines and policies: Yes

No proprietary or copyrighted data misused: I Agree

Notes: No third-party copyrighted code or data was used. The SIIIS/deeplink catalog data is used solely as the competition-provided input for this Theme 2 submission. No secrets or credentials are embedded in the codebase; the Gemini API key is supplied at runtime via environment.

6. Declaration & Sign-Off

I confirm that the AI usage described above is complete and accurate to the best of my knowledge, and that the final submission represents the team's work as classified.

Name of Team Representative: Armaan Tayal

Role: Team Lead

Signature: Armaan Tayal

Date: 29th September, 2026